

PROCESSAMENTO LINGUAGEM NATURAL COM

LLMs / 2025PGS2M1

APRESENTAÇÃO

NOSSAS AULAS SERÃO COM ENCONTROS REMOTOS NAS SEGUINTE DATAS:

24/04 E 09/05 NO HORÁRIO DAS 8:00 ÀS 17:15; TEREMOS UM PERÍODO DE ALMOÇO DE UMA HORA E DOIS INTERVALOS DE 15 MINUTOS.

O CONTEÚDO DAS AULAS SERÃO GRAVADOS E O MATERIAL DISPONIBILIZADO PELO CANVAS;

TODOS OS EXERCÍCIOS SERÃO REALIZADOS DURANTE O TEMPO DOS QUATRO ENCONTROS;

TODAS AS ENTREGAS DEVEM SER FEITAS PELO CANVAS;

EXISTE A POSSIBILIDADE DE TERMOS UMA AULA DE MEIO PERÍODO NO DIA 02/05, CONTUDO ESSA AULA NECESSITA CONFIRMAÇÃO DA COORDENAÇÃO;

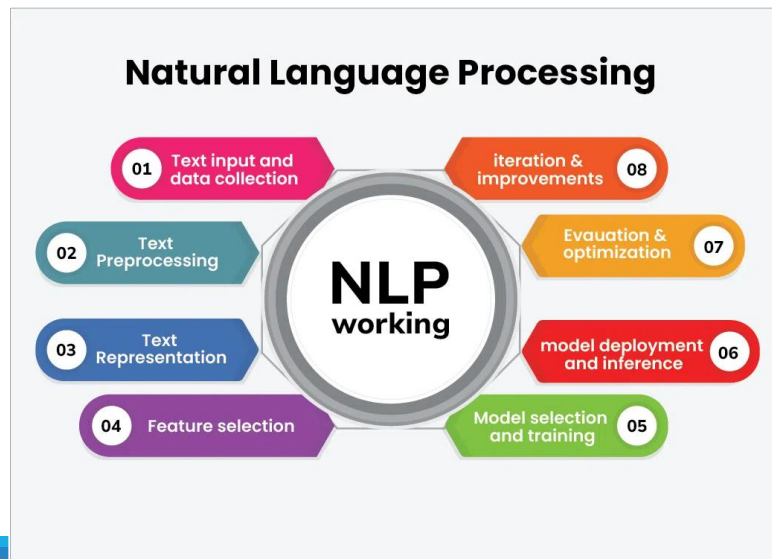
ALÉM DOS EXERCÍCIOS HAVERÁ UMA ENTREGA FINAL.

REVISÃO



REVISÃO

O **Processamento de Linguagem Natural (NLP)** é um campo da **Inteligência Artificial (IA)** que combina **linguística**, **ciência da computação** e **aprendizado de máquina** para permitir que máquinas compreendam, interpretem e gerem linguagem humana (texto ou voz). Seu objetivo é criar sistemas capazes de se comunicar de forma natural com pessoas, automatizando tarefas que dependem da linguagem.



REVISÃO

O Processamento de Linguagem Natural (NLP) desempenha um papel essencial em diversas áreas, destacando-se como uma **ponte entre humanos e máquinas**, pois permite interações mais intuitivas, como conversar com assistentes virtuais (*Alexa, Siri*) ou digitar comandos em linguagem natural. Além disso, viabiliza a **análise de grandes volumes de dados textuais**, extraindo pontos valiosos de redes sociais, avaliações de clientes, documentos médicos ou jurídicos, o que ajuda empresas e instituições a tomarem decisões embasadas.



REVISÃO

Redes Neurais Convolucionais (CNNs): São redes neurais especializadas para processamento de imagens. Elas têm uma arquitetura que explora a estrutura espacial dos dados, fazendo uso de camadas convolucionais, de pooling e totalmente conectadas para extrair características relevantes das imagens.

Redes Neurais Recorrentes (RNNs): São redes neurais projetadas para lidar com dados sequenciais, como texto, áudio e séries temporais. Elas possuem uma estrutura que permite que informações sejam mantidas ao longo do tempo, permitindo a modelagem de dependências temporais complexas.

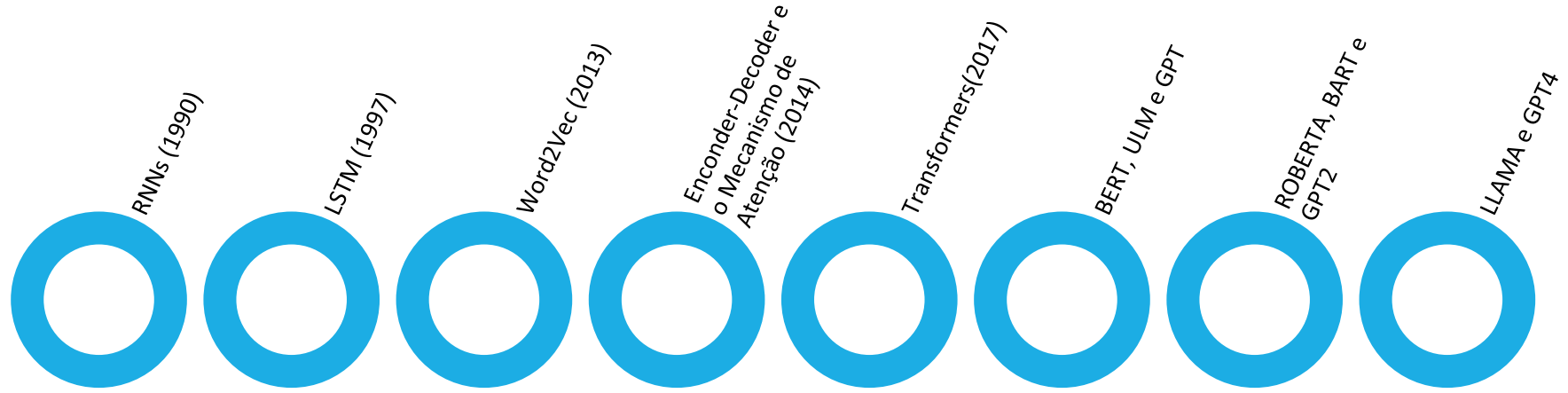
Redes Generativas Adversariais (GANs): São compostas por dois modelos, um gerador e um discriminador, que são treinados de forma adversarial. O gerador cria novas amostras que são indistinguíveis das amostras reais, enquanto o discriminador tenta distinguir entre amostras reais e falsas.

REVISÃO

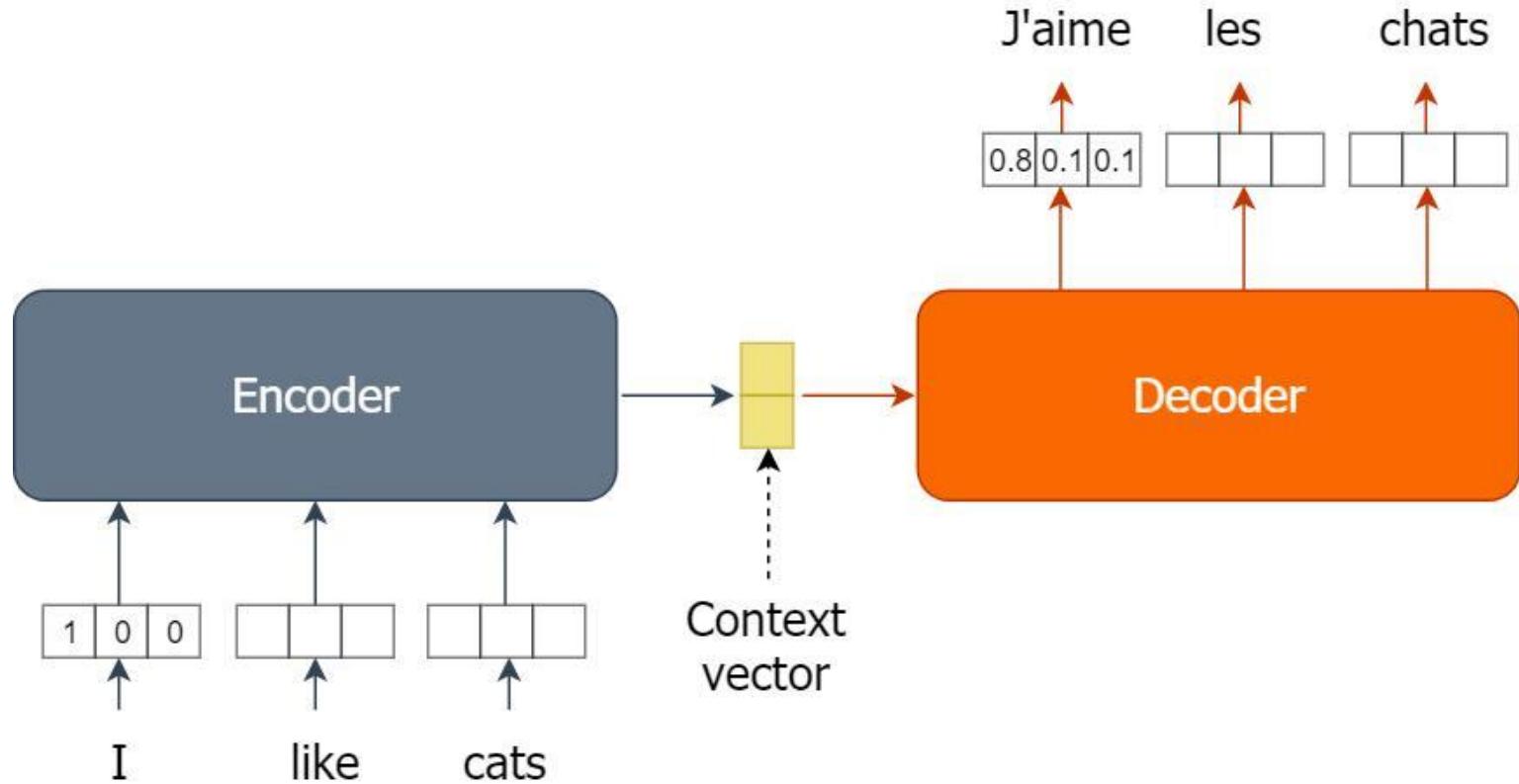
A relação entre Redes Neurais Recorrentes (RNNs) e Large Language Models (LLMs) é de evolução tecnológica e superação de limitações. As RNNs, junto com suas variantes como LSTMs, foram as precursoras no processamento de linguagem natural, enquanto os LLMs modernos utilizam a arquitetura Transformer, que superou as limitações das RNNs para lidar com textos longos e treinamento em larga escala.

O modelo Transformers são um tipo avançado de arquitetura de rede neural, introduzida em 2017, que revolucionou o Processamento de Linguagem Natural (PLN) e IA Generativa. Eles funcionam processando dados sequenciais (como textos) paralelamente, usando um mecanismo de "autoatenção" para entender o contexto e a relação entre palavras distantes, sendo a base de modelos como GPT, BERT e Gemini.

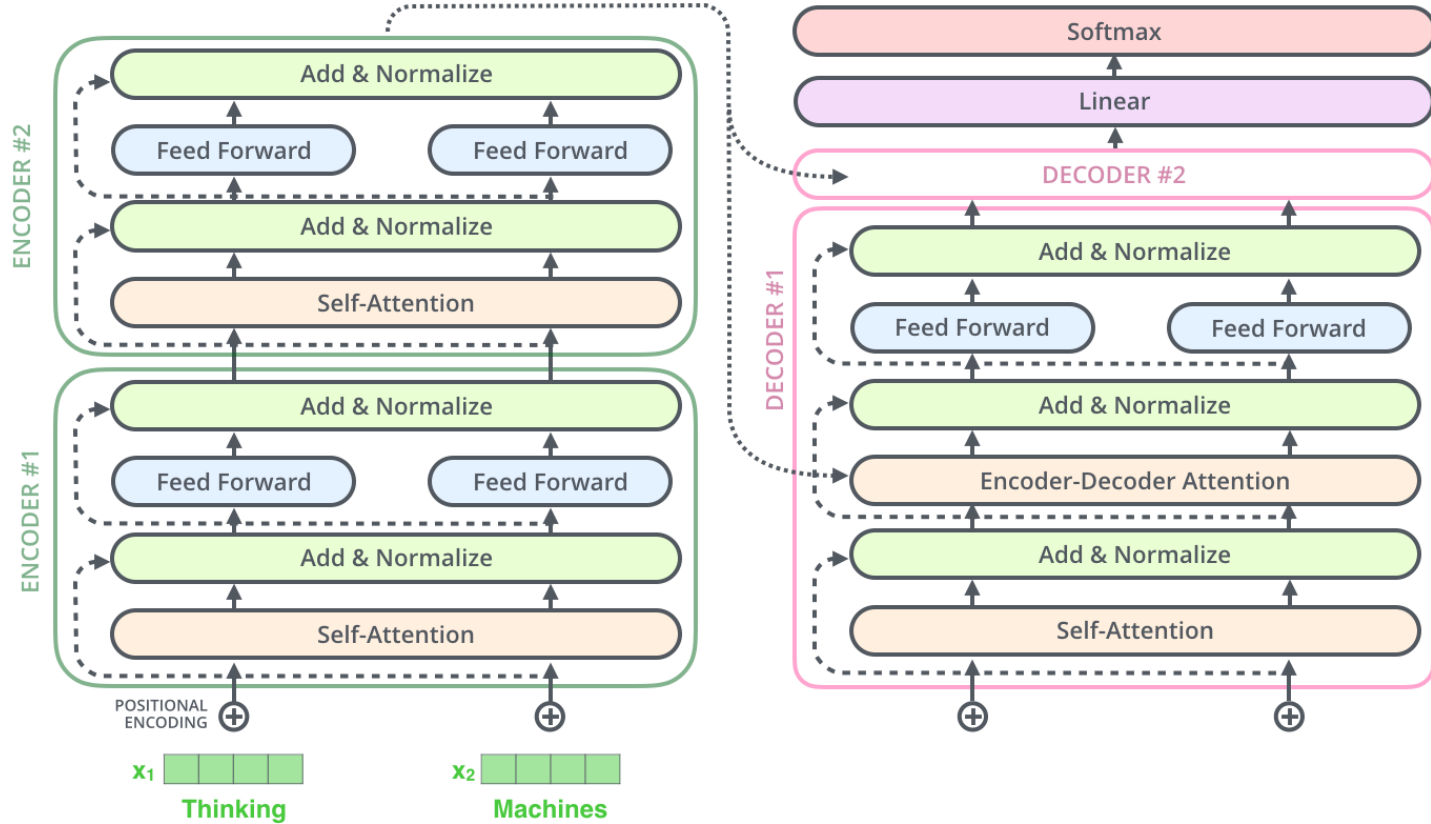
REVISÃO



REVISÃO



REVISÃO



REVISÃO

No próximos slides veremos como funciona uma rede neural sendo utilizada para aplicação de tradução. Todo seu processo de treinamento e aplicação do seu modelo e teste.



REVISÃO

```
1 import numpy as np
2 import tensorflow as tf
3 from tensorflow.keras.models import Model
4 from tensorflow.keras.layers import Input, LSTM, Dense, Embedding
5 from tensorflow.keras.preprocessing.text import Tokenizer
6 from tensorflow.keras.preprocessing.sequence import pad_sequences
7
8 # Base pequena de exemplo
9 entrada_textos = [
10     "hello",
11     "good morning",
12     "good night",
13     "how are you",
14     "thank you",
15     "i love you",
16     "see you later"
17 ]
18
19 saida_textos = [
20     "<start> olá <end>",
21     "<start> bom dia <end>",
22     "<start> boa noite <end>",
23     "<start> como você está <end>",
24     "<start> obrigado <end>",
25     "<start> eu te amo <end>",
26     "<start> até mais tarde <end>"
27 ]
28
```

REVISÃO

```
29 # Tokenização
30 tokenizer_entrada = Tokenizer()
31 tokenizer_saida = Tokenizer(filters='')
32
33 tokenizer_entrada.fit_on_texts(entrada_textos)
34 tokenizer_saida.fit_on_texts(saida_textos)
35
36 entrada_seq = tokenizer_entrada.texts_to_sequences(entrada_textos)
37 saida_seq = tokenizer_saida.texts_to_sequences(saida_textos)
38
39 max_entrada = max(len(seq) for seq in entrada_seq)
40 max_saida = max(len(seq) for seq in saida_seq)
41
42 entrada_seq = pad_sequences(entrada_seq, maxlen=max_entrada, padding="post")
43 saida_seq = pad_sequences(saida_seq, maxlen=max_saida, padding="post")
44
45 vocab_entrada = len(tokenizer_entrada.word_index) + 1
46 vocab_saida = len(tokenizer_saida.word_index) + 1
47
48 # Entrada e saída do decoder
49 decoder_input = saida_seq[:, :-1]
50 decoder_output = saida_seq[:, 1:]
51
```

REVISÃO

```
51
52 # Modelo Encoder-Decoder com LSTM
53 latent_dim = 64
54
55 encoder_inputs = Input(shape=(max_entrada,))
56 encoder_emb = Embedding(vocab_entrada, latent_dim)(encoder_inputs)
57 encoder_lstm, state_h, state_c = LSTM(latent_dim, return_state=True)(encoder_emb)
58
59 decoder_inputs = Input(shape=(max_saida - 1,))
60 decoder_emb = Embedding(vocab_saida, latent_dim)(decoder_inputs)
61 decoder_lstm = LSTM(latent_dim, return_sequences=True)(decoder_emb, initial_state=[state_h, state_c])
62 decoder_dense = Dense(vocab_saida, activation="softmax")(decoder_lstm)
63
64 model = Model([encoder_inputs, decoder_inputs], decoder_dense)
65
66 model.compile(
67     optimizer="adam",
68     loss="sparse_categorical_crossentropy",
69     metrics=["accuracy"]
70 )
71
72 model.fit(
73     [entrada_seq, decoder_input],
74     np.expand_dims(decoder_output, -1),
75     epochs=300,
76     verbose=0
77 )
78
79 print("Modelo RNN/LSTM treinado.")
```

REVISÃO

```
1 def traduzir(frase):
2     entrada = tokenizer_entrada.texts_to_sequences([frase])
3     entrada = pad_sequences(entrada, maxlen=max_entrada, padding="post")
4
5     decoder_seq = tokenizer_saida.texts_to_sequences(["<start>"])[0]
6     decoder_seq = pad_sequences([decoder_seq], maxlen=max_saida - 1, padding="post")
7
8     pred = model.predict([entrada, decoder_seq], verbose=0)
9     pred_indices = np.argmax(pred[0], axis=1)
10
11     palavras = []
12     index_word = tokenizer_saida.index_word
13
14     for idx in pred_indices:
15         palavra = index_word.get(idx, "")
16         if palavra == "<end>":
17             break
18         if palavra not in ["<start>", ""]:
19             palavras.append(palavra)
20
21     return " ".join(palavras)
22
23 print(traduzir("good morning"))
24 print(traduzir("thank you"))
25 print(traduzir("good night"))
26 print(traduzir("world"))
27
```

REVISÃO

Agora veremos como podemos montar um mesmo modelo com uso de Transformers.



REVISÃO

```
1 import numpy as np
2 import tensorflow as tf
3 from tensorflow.keras.models import Model
4 from tensorflow.keras.layers import Input, LSTM, Dense, Embedding
5 from tensorflow.keras.preprocessing.text import Tokenizer
6 from tensorflow.keras.preprocessing.sequence import pad_sequences
7
8 # Base pequena de exemplo
9 entrada_textos = [
10     "hello",
11     "good morning",
12     "good night",
13     "how are you",
14     "thank you",
15     "i love you",
16     "see you later"
17 ]
18
19 saida_textos = [
20     "<start> olá <end>",
21     "<start> bom dia <end>",
22     "<start> boa noite <end>",
23     "<start> como você está <end>",
24     "<start> obrigado <end>",
25     "<start> eu te amo <end>",
26     "<start> até mais tarde <end>"
27 ]
28
```

REVISÃO

```
29 # Tokenização
30 tokenizer_entrada = Tokenizer()
31 tokenizer_saida = Tokenizer(filters='')
32
33 tokenizer_entrada.fit_on_texts(entrada_textos)
34 tokenizer_saida.fit_on_texts(saida_textos)
35
36 entrada_seq = tokenizer_entrada.texts_to_sequences(entrada_textos)
37 saida_seq = tokenizer_saida.texts_to_sequences(saida_textos)
38
39 max_entrada = max(len(seq) for seq in entrada_seq)
40 max_saida = max(len(seq) for seq in saida_seq)
41
42 entrada_seq = pad_sequences(entrada_seq, maxlen=max_entrada, padding="post")
43 saida_seq = pad_sequences(saida_seq, maxlen=max_saida, padding="post")
44
45 vocab_entrada = len(tokenizer_entrada.word_index) + 1
46 vocab_saida = len(tokenizer_saida.word_index) + 1
47
48 # Entrada e saída do decoder
49 decoder_input = saida_seq[:, :-1]
50 decoder_output = saida_seq[:, 1:]
51
```

REVISÃO

```
51
52 # Modelo Encoder-Decoder com LSTM
53 latent_dim = 64
54
55 encoder_inputs = Input(shape=(max_entrada,))
56 encoder_emb = Embedding(vocab_entrada, latent_dim)(encoder_inputs)
57 encoder_lstm, state_h, state_c = LSTM(latent_dim, return_state=True)(encoder_emb)
58
59 decoder_inputs = Input(shape=(max_saida - 1,))
60 decoder_emb = Embedding(vocab_saida, latent_dim)(decoder_inputs)
61 decoder_lstm = LSTM(latent_dim, return_sequences=True)(decoder_emb, initial_state=[state_h, state_c])
62 decoder_dense = Dense(vocab_saida, activation="softmax")(decoder_lstm)
63
64 model = Model([encoder_inputs, decoder_inputs], decoder_dense)
65
66 model.compile(
67     optimizer="adam",
68     loss="sparse_categorical_crossentropy",
69     metrics=["accuracy"]
70 )
71
72 model.fit(
73     [entrada_seq, decoder_input],
74     np.expand_dims(decoder_output, -1),
75     epochs=300,
76     verbose=0
77 )
78
79 print("Modelo RNN/LSTM treinado.")
```

REVISÃO

```
1 def traduzir(frase):
2     entrada = tokenizer_entrada.texts_to_sequences([frase])
3     entrada = pad_sequences(entrada, maxlen=max_entrada, padding="post")
4
5     decoder_seq = tokenizer_saida.texts_to_sequences(["<start>"])[0]
6     decoder_seq = pad_sequences([decoder_seq], maxlen=max_saida - 1, padding="post")
7
8     pred = model.predict([entrada, decoder_seq], verbose=0)
9     pred_indices = np.argmax(pred[0], axis=1)
10
11     palavras = []
12     index_word = tokenizer_saida.index_word
13
14     for idx in pred_indices:
15         palavra = index_word.get(idx, "")
16         if palavra == "<end>":
17             break
18         if palavra not in ["<start>", ""]:
19             palavras.append(palavra)
20
21     return " ".join(palavras)
22
23 print(traduzir("good morning"))
24 print(traduzir("thank you"))
25 print(traduzir("good night"))
26 print(traduzir("world"))
27
```

REVISÃO

Agora veremos como podemos montar um mesmo modelo com uso de Transformers.



REVISÃO

```
1 !pip install transformers sentencepiece -q

1 from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
2
3 model_name = "Helsinki-NLP/opus-mt-en-ROMANCE"
4
5 tokenizer = AutoTokenizer.from_pretrained(model_name)
6 model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
7
8 def traduzir(texto):
9     inputs = tokenizer(texto, return_tensors="pt", padding=True)
10    outputs = model.generate(**inputs)
11    return tokenizer.decode(outputs[0], skip_special_tokens=True)
12
13 print(traduzir("We need your help Starfox!"))
```

REVISÃO

Ao compilar o conteúdo perceba que teremos uma tradução da frase, a mesma atende nossas necessidades?



REVISÃO

```
1 from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
2
3 model_name = "Helsinki-NLP/opus-mt-tc-big-en-pt"
4
5 tokenizer = AutoTokenizer.from_pretrained(model_name)
6 model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
7
8 def traduzir(texto):
9     inputs = tokenizer(texto, return_tensors="pt", padding=True)
10    outputs = model.generate(**inputs)
11    return tokenizer.decode(outputs[0], skip_special_tokens=True)
12
13 print(traduzir("Good morning"))
14 print(traduzir("I love artificial intelligence"))
15 print(traduzir("Eu amo a tecnologia"))
16
```


REVISÃO

Agora com esse novo código qual resultado obtivemos e para o nosso interesse o que seria melhor utilizado em nosso cenário nacional?



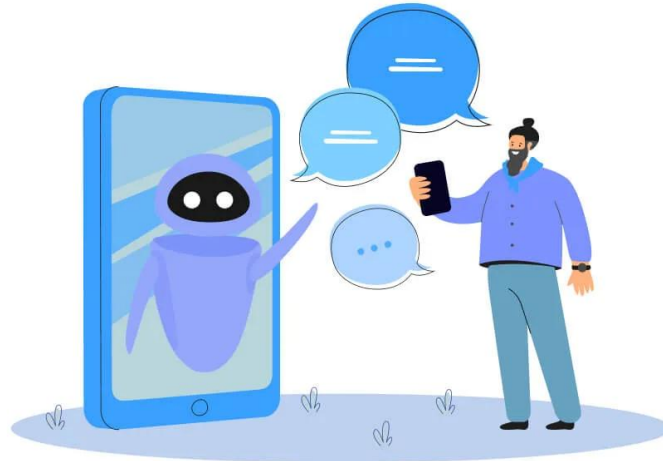
EXERCÍCIO

Acesse seu notebook no Google Colab;

crie um bloco de texto identificando esse exercício e um bloco de código para reproduzir o código de exemplo;

Após escrever o código compile para ver o resultado analisado na frase de teste;

Ao término, utilize o código em português e insira um trecho de uma música que conheça e veja se o mesmo traduz de forma correta. Coloque um trecho longo da música e a mesma deve ser em inglês.



REVISÃO

TÉCNICA	DESCRIÇÃO	APLICAÇÃO EM CHATBOTS
TOKENIZAÇÃO	DIVIDIR FRASES EM PALAVRAS OU SENTENÇAS	SEPARAR COMANDOS DO USUÁRIO PARA ANÁLISE
STEMMING	REDUZIR PALAVRAS À SUA RAIZ (EXEMPLO: “CORRENDO” – “CORRER”)	MELHORAR A EFICIÊNCIA DO MODELO DE CHATBOT
LEMATIZAÇÃO	SIMILAR AO STEMMING, MAS MANTENDO SIGNIFICADO GRAMATICAL CORRETO	INTERPRETAR MELHOR A INTENÇÃO DO USUÁRIO
RECONHECIMENTO DE ENTIDADES (NER)	IDENTIFICAR NOMES, DATAS, LOCAIS EM UM TEXTO	EXTRAIR INFORMAÇÕES DE MENSAGENS DO USUÁRIO
POS TAGGING (PART-OF-SPEECH TAGGING)	IDENTIFICA A FUNÇÃO GRAMATICAL DAS PALAVRAS	MELHOR INTERPRETAÇÃO DE PERGUNTAS COMPLEXAS EM UM TRANSAÇÕES
TF-IDF (TERM FREQUENCY – INVERSE DOCUMENT FREQUENCY)	MEDE A IMPORTANCIA DE PALAVRAS DENTRO DE UM CONJUNTO DE TEXTOS	MELHORAR RESPOSTAS DE CHATBOTS A PERGUNTAS SOBRE DOCUMENTOS LONGOS

REVISÃO

Principais Passos do Pré-processamento de Texto

- **Tokenização:** Dividir o texto em unidades menores chamadas tokens, que podem ser palavras, frases, subpalavras ou até caracteres. *Exemplo: A frase "Olá! Como você está?" é tokenizada em ["Olá", "!", "Como", "você", "está", "?"].*
- **Stemming:** Reduzir palavras à sua raiz aproximada, mesmo que não seja uma palavra válida (ex.: "correndo" → "corr").
- **Lemmatização:** Reduzir palavras à sua forma base (lema) usando regras linguísticas (ex.: "correndo" → "correr").

REVISÃO

Análise Morfológica

- **Part-of-Speech Tagging (POS):** Identifica a classe gramatical de cada palavra (substantivo, verbo, adjetivo, etc.). *Exemplo:* Na frase "**O gato preto correu rápido**", as tags seriam: "**gato**" → substantivo, "**preto**" → adjetivo, "**correu**" → verbo.

Exemplo Prático

Frase: "A Amazon anunciou hoje um novo serviço em Londres, que custará US\$ 10 por mês."

Análise Morfológica (Part-Of-Speech):

"Amazon" → substantivo próprio, "anunciou" → verbo, "hoje" → advérbio, "Londres" → substantivo próprio.

REVISÃO

Análise Semântica

- **Named Entity Recognition (NER):** Identifica entidades nomeadas, como pessoas, organizações, datas e locais. *Exemplo:* Em "**João trabalha na Google em São Paulo desde 2020**", reconhece: "**João**" (pessoa), "**Google**" (organização), "**São Paulo**" (local), "**2020**" (data).
- **Resolução de Referência:** Determina a qual entidade pronomes ou expressões se referem. *Exemplo:* Em "**Ana comprou um livro. Ela adorou.**", "**Ela**" refere-se a "**Ana**".

Exemplo Prático

Frase: "A Amazon anunciou hoje um novo serviço em Londres, que custará US\$ 10 por mês."

Análise Semântica (Named Entity Recognition):

Entidades: "Amazon" (organização), "Londres" (local), "US\$ 10" (valor monetário).

REVISÃO

O TF-IDF (Frequência do Termo - Frequência Inversa do Documento) é uma técnica estatística usada em Processamento de Linguagem Natural (NLP) para análise de textos. Ele mede a importância de uma palavra em um documento, considerando sua frequência no documento e no conjunto de documentos.

A técnica é utilizada para classificação de textos (Exemplo: detectar spam em e-mails), busca e recuperação de informações (Exemplo: motores de busca como Google). Extração de palavras-chave e representação numérica de textos para Machine Learning. O TF-IDF é calculado utilizando dois valores e a seguinte formula:

$$TF-IDF(t) = TF(t) \times IDF(t)$$

Contudo para encontrarmos os valores devemos utilizar as seguintes formulas utilizadas no slide a seguir:

REVISÃO

A primeira é a **TF (Term Frequency - Frequência do Termo)** que mede quantas vezes um termo aparece em um documento e se uma palavra aparece muitas vezes em um único documento, ela pode ser relevante. Sua formula é:

$$TF(t) = \frac{\text{Número de vezes que o termo aparece no documento}}{\text{Total de termos no documento}}$$

Já o IDF (Inverse Document Frequency - Frequência Inversa do Documento) e mede o quão raro é um termo em um conjunto de documentos. Palavras comuns como "o", "e", "de" aparecem em quase todos os textos, então seu IDF será baixo e se um termo aparece poucas vezes no conjunto total de documentos, ele pode ser mais relevante. Sua formula é:

$$IDF(t) = \log \left(\frac{\text{Número total de documentos}}{\text{Número de documentos que contêm o termo } t} \right)$$

REVISÃO

Exemplo prático:

Vamos supor que temos **três documentos**:

- Doc 1**: "O chatbot é inteligente e responde bem."
- Doc 2**: "O chatbot usa Machine Learning para aprender."
- Doc 3**: "Machine Learning é importante para IA."

Agora, queremos calcular o **TF-IDF da palavra "chatbot"**.

Frequência do Termo (TF)

Palavra	Doc 1 (TF)	Doc 2 (TF)	Doc 3 (TF)
chatbot	1/6	1/6	0/6
Machine	0/6	1/6	1/6

REVISÃO

Frequência Inversa do Documento (IDF) A palavra "chatbot" aparece em **2 de 3** documentos:

$$IDF(chatbot) = \log(3/2) = 0.176$$

Cálculo do TF-IDF

Palavra	Doc 1 (TF-IDF)	Doc 2 (TF-IDF)	Doc 3 (TF-IDF)
chatbot	0.167×0.176	0.167×0.176	0×0.176

Se um termo tem **TF-IDF alto**, significa que ele é **relevante** para aquele documento.

REVISÃO

Podemos calcular TF-IDF automaticamente usando a biblioteca TfidfVectorizer do scikit-learn. Lembrando que o Scikit-Learn é uma biblioteca de Machine Learning para Python, construída sobre o NumPy, SciPy e Matplotlib. Ele é amplamente usado para tarefas de aprendizado de máquina supervisionado e não supervisionado, incluindo classificação, regressão, clusterização e redução de dimensionalidade.

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 # Criando os documentos
4 documentos = [
5     "O chatbot é inteligente e responde bem.",
6     "O chatbot usa Machine Learning para aprender.",
7     "Machine Learning é importante para IA."
8 ]
9
10 # Criando o modelo TF-IDF
11 vectorizer = TfidfVectorizer()
12 tfidf_matrix = vectorizer.fit_transform(documentos)
13
14 # Mostrando os termos e suas pontuações
15 print(vectorizer.get_feature_names_out()) # Lista de palavras analisadas
16 print(tfidf_matrix.toarray()) # Matriz TF-IDF dos documentos
17
```

REVISÃO

Dentro do resultado obtido temos a lista de palavras (vocabulário), essa lista de palavras analisadas pelo TF-IDF, foi construído a partir dos documentos. Palavras muito comuns foram removidas automaticamente pelo TfidfVectorizer.

Sua matriz, tem cada linha representando um documento, cada coluna representando uma palavra do vocabulário. Os valores indicam o peso do TF-IDF para aquela palavra no documento correspondente.

```
['aprender' 'bem' 'chatbot' 'ia' 'importante' 'inteligente' 'learning'
 'machine' 'para' 'responde' 'usa']
[[0.          0.52863461 0.40204024 0.          0.          0.52863461
  0.          0.          0.          0.52863461 0.          ]
 [0.48148213 0.          0.36617957 0.          0.          0.
  0.36617957 0.36617957 0.36617957 0.          0.48148213]
 [0.          0.          0.          0.51741994 0.51741994 0.
  0.3935112 0.3935112 0.3935112 0.          0.          ]]
```

REVISÃO

Analisando os pesos das palavras nos documentos podemos observar a tabela a seguir:

Palavra	Documento 1	Documento 2	Documento 3
aprender	0.0	0.0	0.0
bem	0.5286	0.0	0.0
chatbot	0.4020	0.0	0.0
ia	0.0	0.4814	0.0
importante	0.0	0.0	0.5174
inteligente	0.0	0.3661	0.0
learning	0.5286	0.3661	0.5174
machine	0.0	0.3661	0.5174
para	0.0	0.4814	0.0
responde	0.5286	0.0	0.0
usa	0.0	0.0	0.0

Palavras como “learning” e “machine” aparecem com frequencia, por isso seus pesos não são altos, em contrapartida palavras mais únicas em um documento recebem valores maiores.

REVISÃO

O pré-processamento é a **base para qualquer pipeline de NLP**, transformando texto caótico em dados estruturados e prontos para alimentar modelos de aprendizado de máquina ou análises linguísticas. O uso do pré-processamento tem como essência atender os seguintes pontos:

- **Reduz ruídos:** Textos brutos podem conter elementos irrelevantes (como emoticons ou códigos) que atrapalham a análise.
- **Padroniza dados:** Garante que palavras semelhantes (ex.: "Casa" e "casa") sejam tratadas igualmente.
- **Melhora eficiência:** Reduz a complexidade computacional ao eliminar dados redundantes.
- **Aumenta precisão:** Evita que modelos sejam confundidos por variações superficiais (ex.: plural vs. singular).

REVISÃO

Criar um chatbot que detecta o **tom emocional de uma mensagem** (positivo, neutro ou negativo). Para isso, os alunos podem usar **Análise de Sentimento** com NLP.

```
1 #INSTALAÇÃO DAS BIBLIOTECAS
2 !pip install deep_translator textblob
3
4 #IMPORTAÇÃO DAS BIBLIOTECAS
5 from deep_translator import GoogleTranslator
6 from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
7
8 #INICIALIZAÇÃO DO ANALISADOR DE SENTIMENTO
9 analisador = SentimentIntensityAnalyzer()
10
11 #DEFINIÇÃO DA FUNÇÃO DE ANÁLISE DE SENTIMENTO
12 def analisar_sentimento_vader(texto):
13     texto_ingles = GoogleTranslator(source='auto', target='en').translate(texto)
14
15     #CÁLCULO DO SENTIMENTO
16     sentimento = analisador.polarity_scores(texto_ingles)
17
18     #CLASSIFICAÇÃO DO SENTIMENTO
19     if sentimento['compound'] >= 0.05:
20         return "Positivo"
21     elif sentimento['compound'] <= -0.05:
22         return "Negativo"
23     else:
24         return "Neutro"
25
26 # TESTE DA FUNÇÃO
27 mensagem = "Estou muito feliz com o serviço do chatbot!"
28 print(analisar_sentimento_vader(mensagem))
```

REVISÃO

Este modelo criado utiliza o **VADER** (*Valence Aware Dictionary and sEntiment Reasoner*) este é um **analisador de sentimentos baseado em regras e léxico**, projetado especificamente para interpretar **textos informais** como postagens em redes sociais, comentários de usuários e mensagens curtas.

Suas vantagens são:

De ser um otimizado para textos informais, no qual funciona bem para frases curtas, capturando nuances emocionais sem necessidade de treinamento. Leva em conta contexto e intensificadores, como emojis, letras maiúsculas, negações e exclamações. Classifica textos em Positivo, Negativo ou Neutro, atribuindo um **score compound** que indica a polaridade geral do texto. Além de ser baseado em um dicionário pré-treinado, o que o torna rápido e eficiente, sem precisar de modelos de machine learning.

EXERCÍCIO

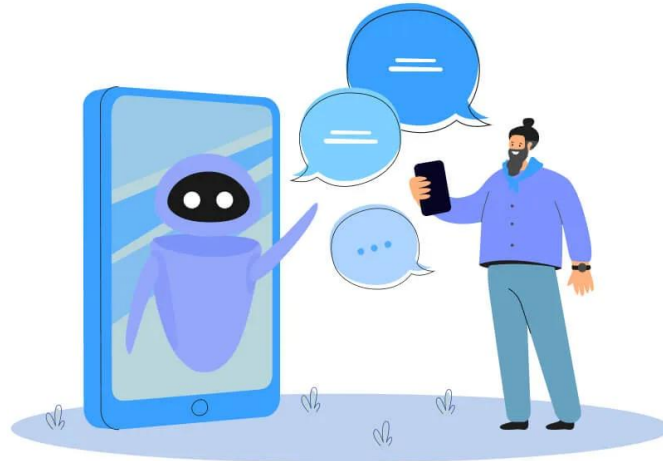
Acesse seu notebook no Google Colab;

crie um bloco de texto identificando esse exercício e um bloco de código para reproduzir o código de exemplo;

Após escrever o código compile para ver o resultado analisado na frase de teste;

Ao término insira mais dois exemplos um de frase neutra e outra com frase negativa;

Teste novamente e veja o resultado.

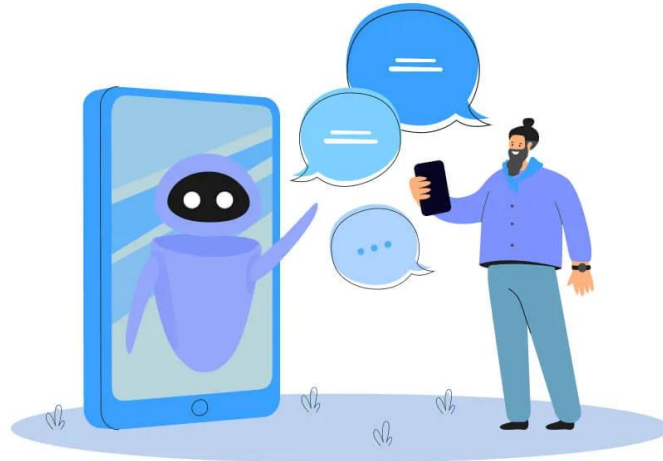


EXERCÍCIO

Observando o que foi aprendido, como um tradutor serviria em uma aplicação no contexto do seu trabalho?

Que utilizaria está aplicação?

Para responder essas perguntas, crie um bloco de texto no seu Notebook e responda dentro do conteúdo.



REVISÃO

Agora analisaremos o nosso modelo para compreender a profundidade de conhecimento que o mesmo tem. Para isso faremos um processo diferente, foi acessado uma loja virtual com um produto e foi selecionado três comentários. Um com cinco estrelas, o segundo com três estrelas e o último com uma estrela.

★★★★★

23 set. 2024

Jogoço, tenho 32 anos e gostei muito, minha namorada tbm se divertiu, o jogo atende tanto a iniciantes por ser bem instrutivo como a fãs de video games por ter muitas horas de jogo e alguns níveis difíceis. Quem gosta dos visuais fofinhos tbm não vai se decepcionar com a possibilidade de usar várias roupas com o mário e sua habilidade de capturar outros bixinhos.

É útil  1

⋮

★★★★☆

03 ago. 2023

Tudo ok mas parece que o jogo já foi usado pois os pontos de ouro pra resgate já foi resgatado antes de mim... alguém já usou antes.

É útil  4

⋮

★☆☆☆☆

04 mar. 2024

Produto não veio em português como estava na descrição do produto.

É útil  0

⋮

REVISÃO

Com os textos inserimos o resultado não foi o esperado.

Positivo
Positivo
Neutro

Conforme podemos observar existe uma falha na identificação do segundo texto e do terceiro. O VADER usa a métrica ***compound*** para determinar se um texto é positivo, negativo ou neutro com base nos seguintes valores padrões:

INTERVALO COMPOUND	CLASSIFICAÇÃO PADRÃO
≥ 0.05	POSITIVO
≤ -0.05	NEGATIVO
$-0.05 < \text{COMPOUND} < 0.05$	NEUTRO

REVISÃO

Uma solução para melhorar o desempenho é diminuir esse limiar para capturar mais nuances na emoção do texto, para isso podemos mudar a função de análise:

```
# Ajustando os thresholds para melhorar a sensibilidade
if sentimento['compound'] >= 0.02: # Antes era 0.05
    return "Positivo"
elif sentimento['compound'] <= -0.02: # Antes era -0.05
    return "Negativo"
else:
    return "Neutro"
```

Outra solução seria aplicar um dicionário personalizado para melhorar a análise, recurso que auxiliaria o VADER com pontuações pré-definidas para calcular a polaridade.

```
# Ajuste de pesos para palavras específicas
analizador.lexicon.update({
    "jogoço": 2.5,
    "divertiu": 1.5,
    "resgatado": -3.0,
    "usado": -3.0,
    "fraco": -2.0,
    "desapontado": -2.5,
    "decepcionante": -3.0
})
```

REVISÃO

Apesar das soluções propostas ainda podemos esbarrar na complexidade do uso de um tradutor devido o VADER não ter o foco da língua portuguesa. Além disso, conforme vimos os comentários escolhidos contem textos longos outra deficiência do modelo.

Desta forma, podemos considerar outras duas boas opções de aplicação. A primeira seria o BERTimbau (modelo de NLP baseado em BERT treinado para português) ou modelos da **Hugging Face**. Já o segundo seria a **API Sentiment Analysis da Hugging Face**, que suporta múltiplos idiomas, incluindo português. Antes de apresentar os modelos devemos conhecer o que é a Hugging Face.



MODELOS TRANSFORMERS E LLM

O Hugging Face é uma empresa e uma plataforma open-source especializada em modelos de inteligência artificial para Processamento de Linguagem Natural (NLP) e outras tarefas de aprendizado profundo. Ele oferece uma biblioteca chamada Transformers, que permite facilmente usar e treinar modelos pré-treinados como BERT, GPT, T5, DistilBERT, e muitos outros.



Hugging Face

REVISÃO

Primeiro faremos nossos testes com a **API Sentiment Analysis**, que suporta múltiplos idiomas, incluindo português. Suporta vários idiomas, incluindo português. Porém pode ser mais lento, pois usa redes neurais profundas.

```
1 from transformers import pipeline
2
3 # Criar pipeline de análise de sentimento multilíngue
4 analise_sentimento = pipeline("sentiment-analysis", model="cardiffnlp/twitter-xlm-roberta-base-sentiment")
5
6 # Teste
7 mensagem1 = "Jogo, tenho 32 anos e gostei muito, minha namorada tbm se divertiu, o jogo atende tanto a iniciantes por ser bem instrutivo como a fãs
8 resultado1 = analise_sentimento(mensagem1)
9 print(resultado1)
10
11 mensagem2 = "Tudo ok mas parece que o jogo já foi usado pois os pontos de ouro pra resgate já foi resgatado antes de mim... alguém já usou antes."
12 resultado2 = analise_sentimento(mensagem2)
13 print(resultado2)
14
15 mensagem3 = "Produto não veio em português como estava na descrição do produto."
16 resultado3 = analise_sentimento(mensagem3)
17 print(resultado3)
18
```


REVISÃO

Neste exemplo, temos pontos importantes que são:

- `pipeline("sentiment-analysis")` → Carrega automaticamente um modelo de análise de sentimentos.
- `"cardiffnlp/twitter-xlm-roberta-base-sentiment"` → Modelo baseado em XLM-RoBERTa, treinado em tweets para detectar sentimentos em diversos idiomas, incluindo português.
- As saídas esperadas deste modelo são de três categorias: `LABEL_0` → Negativo; `LABEL_1` → Neutro; `LABEL_2` → Positivo
- Como podemos ver o modelo fez uma boa classificação do primeiro e o segundo comentário, contudo pecou no último deixando de forma neutra.

```
Device set to use cpu
[{'label': 'positive', 'score': 0.7302335500717163}]
[{'label': 'neutral', 'score': 0.5551769137382507}]
[{'label': 'neutral', 'score': 0.5355969071388245}]
```

REVISÃO

Agora faremos nossos testes com o **BERTimbau** que é um modelo de Processamento de Linguagem Natural (NLP) **baseado no BERT**, treinado **especificamente para português**. Em sua proposta o BERTimbau melhora a interpretação de textos em português. Pode ser usado para análise de sentimento, perguntas e respostas e classificação de textos e utiliza Transformers, uma das arquiteturas mais poderosas para NLP.

```
1 #IMPORTAÇÃO DO PIPELINE
2 from transformers import pipeline
3
4 # CARREGA O MODELO ESCOLHIDO
5 analise_sentimento = pipeline("sentiment-analysis", model="nlp-town/bert-base-multilingual-uncased-sentiment")
6
7 # TESTE COM TRÊS FRASES
8 mensagem1 = "Jogo, tenho 32 anos e gostei muito, minha namorada tbm se divertiu, o jogo atende tanto a iniciantes por ser bem instrutivo como a fãs
9 resultado1 = analise_sentimento(mensagem1)
10 print(resultado1)
11
12 mensagem2 = "Tudo ok mas parece que o jogo já foi usado pois os pontos de ouro pra resgate já foi resgatado antes de mim... alguém já usou antes."
13 resultado2 = analise_sentimento(mensagem2)
14 print(resultado2)
15
16 mensagem3 = "Produto não veio em português como estava na descrição do produto."
17 resultado3 = analise_sentimento(mensagem3)
18 print(resultado3)
```

REVISÃO

Neste exemplo, temos pontos importantes que são:

- O comando **pipeline("sentiment-analysis")** é uma ferramenta simplificada para carregar modelos de NLP já treinados.
- O modelo "nlptown/bert-base-multilingual-uncased-sentiment" é um modelo multilíngue, treinado em textos de avaliações (reviews), funcionando bem para classificar sentimentos.
- O modelo carrega um classificador de sentimentos pré-treinado baseado em BERT, especializado na análise de textos multilíngues.

```
4 # CARREGA O MODELO ESCOLHIDO
5 analyse_sentimento = pipeline("sentiment-analysis", model="nlptown/bert-base-multilingual-uncased-sentiment")
6
```

REVISÃO

- Esse modelo não classifica apenas como "positivo, negativo ou neutro", mas retorna um score de 1 a 5 estrelas, sendo: 1 estrela → Muito negativo; 2 estrelas → Negativo; 3 estrelas → Neutro; 4 estrelas → Positivo; 5 estrelas → Muito positivo. Se observarmos o valor informado de saída o modelo conseguiu alcançar uma classificação interessante.

```
Device set to use cpu
```

```
[{'label': '4 stars', 'score': 0.5159031748771667}]
```

```
[{'label': '3 stars', 'score': 0.5512217283248901}]
```

```
[{'label': '1 star', 'score': 0.581873893737793}]
```

EXERCÍCIO

Procure um produto de uma loja virtual, veja se o produto tem comentários, positivos, neutros e negativos;

Copie os textos de cada um dos comentários e o link da página do produto;

Acesse seu notebook no Google Colab;

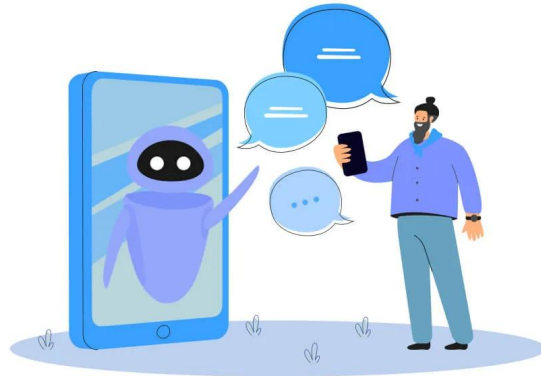
Crie um bloco de texto identificando esse exercício e adicione os comentários e o link da página;

Crie o primeiro código com a **API Sentiment Analysis** e depois com o modelo **BERTimbau**;

Utilize os comentários escolhidos para o teste e aplique em ambos os modelos;

Veja os resultados obtidos;

Não serão aceitas atividades com produtos e comentários iguais.

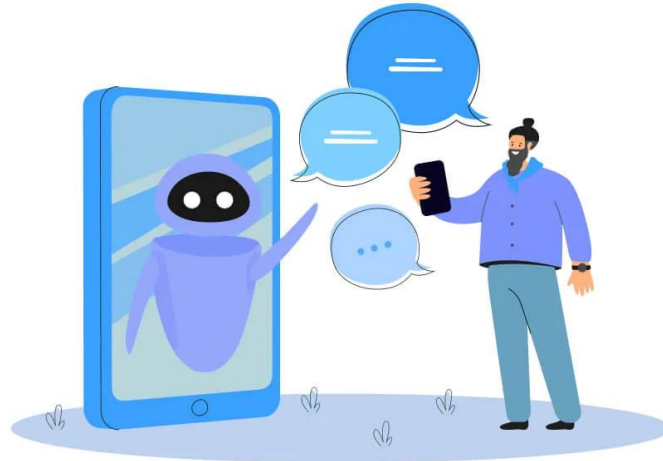


EXERCÍCIO

Observando o que foi aprendido, como um tradutor serviria em uma aplicação no contexto do seu trabalho?

Que utilizaria está aplicação?

Para responder essas perguntas, crie um bloco de texto no seu Notebook e responda dentro do conteúdo.



DÚVIDAS ou PERGUNTAS?



MUITO OBRIGADO!!!!



daniel.ohata@facens.br

REFERÊNCIAS

BISONG, Ekaba; BISONG, Ekaba. Google colabatory. **Building machine learning and deep learning models on google cloud platform: a comprehensive guide for beginners**, p. 59-64, 2019.

DE ANDRADE, Amanda Figueiredo et al. A ÉTICA NO USO DA INTELIGÊNCIA ARTIFICIAL E SEUS RISCOS JURÍDICOS. *Revista Acadêmica Online*, v. 11, n. 56, p. e1400-e1400, 2025.

ISZCZUK, Ana Claudia Duarte et al. Evoluções das tecnologias da indústria 4.0: dificuldades e oportunidades para as micro e pequenas empresas. **Brazilian Journal of Development**, v. 7, n. 5, p. 50614-50637, 2021.

MARCONI, Francesco. Artificial Intelligence Backbone. < <https://twitter.com/fpmarconi/status/794208040207740928> >, 2016 Acesso 01/01/2019.

MICHALSKI, Ryszard Stanislaw; CARBONELL, Jaime Guillermo; MITCHELL, Tom M. (Ed.). **Machine learning: An artificial intelligence approach**. Springer Science & Business Media, 2013.

ROSÁRIO, João Maurício. **Automação industrial**. Editora Baraúna, 2012.

RUSSELL, Stuart J.; NORVIG, Peter. **Artificial intelligence: a modern approach**. Malaysia; Pearson Education Limited, 2016.

SANTOS, Gustavo Soares. Novas Tecnologias Aplicadas na Construção Civil: Conceitos da Indústria 4.0. *RCT-Revista de Ciência e Tecnologia*, v. 8, 2022.

ZHOU, Zhi-Hua. **Machine learning**. Springer nature, 2021.